

Formalization and Tool Support for STPA

A literature review positioning a coverage-typing checker over the analysis artifact graph

Claude Opus 4.8 (Anthropic)

June 24, 2026

Abstract

This review surveys existing work on formalizing System-Theoretic Process Analysis (STPA) and the openly available toolchains that support it, in order to locate the niche occupied by a type-system / Datalog “coverage checker” over the STPA *analysis artifact graph*. We find that the relevant prior work clusters into three lineages—a mainstream requirements-formalization thread, a logic-programming thread, and an open-tooling thread—none of which frames orphaned Unsafe Control Actions (UCAs) as a totality (non-exhaustiveness) error or treats the loss $\rightarrow$ hazard $\rightarrow$ UCA $\rightarrow$ constraint chain as a typing judgment. The constituent techniques are all mature in adjacent rooms; assembling them for STPA appears to be an open contribution.

1 Scope and the question being asked

A distinction runs through this review. STPA produces two things that are easy to conflate:

1. a model of the *controlled process* — the control structure, its control actions, feedback, and process-model variables; and
2. an *analysis document* — the chain of losses, hazards, system-level constraints, UCAs, controller constraints/requirements, and loss scenarios, together with the traceability links among them.

Almost all existing formalization effort verifies properties of (1): does the design satisfy the safety constraints? The question motivating this review is a property of (2): does *every* UCA propagate down to at least one controller constraint or requirement, and does every artifact cite a well-kinded parent? That is a *coverage* or *totality* property of the document, not a behavioral property of the system. The reframing is that an orphaned UCA — one with no derived requirement — is exactly a non-exhaustive-match error, and the artifact chain is a typing judgment.

2 Lineage 1: requirements formalization (Thomas, Abdulkhaleq, Leveson)

This is the dominant and most mature thread, and it runs *downstream* of the coverage question. Thomas’s MIT dissertation introduced context tables that systematize UCA identification by enumerating combinations of process-model variables. Abdulkhaleq and Wagner operationalized this: the XSTPA plug-in implements Thomas’s context tables to systematize and partially automate UCA identification. From the resulting hazardous rules, the pipeline derives refined safety constraints and then translates them automatically into Linear Temporal Logic (LTL) specifications, which a model checker (SPIN, NuSMV) verifies against a manually constructed safe behavioral model;

the same artifacts drive automatic safety-based test-case generation (the STPA TCGenerator and STPA Verifier plug-ins).

The key observation for positioning: this lineage formalizes *controlled-process behavior*. It asks whether the design honors a constraint, expressed in temporal logic. It maintains traceability links as a matter of bookkeeping, but it does not treat “every UCA has a constraint” as a statically verified totality property of the analysis graph. The LTL is about the system; the coverage question is about the document. These are orthogonal, and the coverage question is logically prior — you cannot verify a constraint that was never written because its UCA was orphaned.

3 Lineage 2: logic programming over STPA (Innes et al.)

The nearest neighbor in spirit is Innes, Ireland, Lin, and Ramamoorthy, *Anticipating Accidents through Reasoned Simulation* (TAS ’23). They state the motivating defect plainly: STPA is systematic but relatively informal, and in consequence offers no mechanized support for catching inconsistencies and invalid assumptions as they enter the analysis. Their remedy encodes UCAs and constraints in Epistemic Logic Programs (ELP), an Answer-Set-Programming-family formalism, to obtain mechanized consistency proofs, and then bridges the formalized predicates to a CARLA driving simulator to explore loss scenarios.

This is the closest the open literature comes to the present idea, and the overlap is real: a declarative logic substrate applied to STPA artifacts. But the *target* differs. Innes et al. aim at causal-scenario semantics and simulation grounding — reasoning about *why* a UCA occurs and animating it — rather than at a coverage/well-kindness judgment over the artifact DAG. They also use ELP (epistemic, disjunctive, with worldview semantics) rather than the much weaker Datalog that the coverage checks actually require; coverage and orphan detection need only stratified negation over a closed document, not epistemic operators. Notably, the authors flag that the tooling to make even their approach practical does not yet exist: assistance is needed to bridge STPA to ELP and ELP to the simulator, and they regard dedicated tool support as essential and not yet built.

4 Lineage 3: open toolchains (data model and change propagation)

XSTAMPP. The de-facto open platform is XSTAMPP (Eclipse Rich Client Platform, open source, SE-Stuttgart on GitHub), the successor to A-STPA. Its plug-in suite includes XSTPA (context tables), the STPA Verifier (model checking), and the STPA TCGenerator (test generation). For the coverage question, two facts about XSTAMPP matter. First, it already performs the *bookkeeping* half of the idea: changes are automatically linked across analysis steps, so updates to the control structure or hazards propagate to related artifacts, and the tool is explicitly intended to help analysts avoid consistency defects. Second, and most useful, XSTAMPP keeps an internal XML representation of every STPA component specifically to enable integration with other tools. That XML is the natural source from which to extract Datalog facts (`uca/1`, `issues/2`, `refines/2`, `uca.traces/2`, ...), which means a coverage checker can be a *layer over* XSTAMPP rather than a from-scratch editor.

STPAmaster. The commercial STPAmaster advertises an automated check for common mistakes and analysis consistency, derived from the STPA handbook, plus change management that updates the analysis as the control structure, UCAs, and context tables change. This is the closest any shipping tool comes to the coverage idea — but it is closed-source, and the check set is not

published as a formal, inspectable rule base, so it cannot be evaluated as a type system or extended declaratively.

What the tools do not do. Neither XSTAMPP nor STPAmaster frames coverage as a *type system*, exposes the checks as a declarative rule set one can read and extend, or distinguishes (in a checkable way) the acyclic authority relation from the cyclic information-flow relation. Change propagation keeps links *current*; it does not *prove* that the links form a total covering of UCAs by requirements.

5 Adjacent techniques: completeness/consistency in requirements and ontologies

The machinery the coverage checker needs is mature in neighboring fields. Ontology-based approaches to requirements specification use reasoning for consistency checking and rule-driven completeness checking, and tools such as OntoCheck enforce metadata completeness and naming conventions over OWL artifacts. More directly, goal-directed Answer-Set-Programming work on dynamic consistency checking uses precisely the reachability idiom that the coverage query needs — a recursive **reachable/1** predicate over graph edges with a denial constraint requiring every vertex to be reachable. That denial, “every vertex must be reachable,” is structurally identical to “every UCA must derive a requirement.” The technique exists; it has simply not been pointed at the STPA artifact graph.

6 The gap

Summarizing the landscape against the coverage-typing idea:

- **Traceability as a DAG** — standard practice across the field; not novel.
- **Change propagation over the artifact graph** — done by XSTAMPP and STPAmaster; keeps links current but does not verify totality.
- **Logic programming over STPA** — done by Innes et al., but targeting scenario semantics and simulation, with ELP rather than Datalog, and with the artifact-graph coverage judgment out of scope.
- **Behavioral verification of constraints** — the Thomas/Abdulkhaleq LTL pipeline; downstream of coverage and orthogonal to it.
- **Completeness/consistency via ASP reachability and ontologies** — mature in requirements engineering generally, not applied to STPA.

What is absent is the assembly: a typed / Datalog coverage checker over the STPA artifact graph that (i) separates the acyclic authority relation from the cyclic information-flow relation, (ii) treats orphaned UCAs and dead requirements as totality (non-exhaustiveness) diagnostics, and (iii) pairs a whole-document static pass with construction-time smart constructors in a structured editor. Each ingredient lives in an adjacent room; combining them for STPA appears to be an open and publishable contribution.

7 Suggested next steps

1. Obtain the full Innes et al. paper (arXiv:2306.04499) to position precisely on the logic-programming axis and to contrast ELP’s expressivity against the weaker Datalog the coverage checks require.
2. Inspect the XSTAMPP XML schema (SE-Stuttgart/XSTAMPP on GitHub) to measure how close its data model already is to the intended fact base, and to scope an export-to-Datalog adapter.
3. Map the schema onto the typing rules from the companion document (*A Type System for STPA*), implementing the coverage and orphan rules as a Datalog/ASP layer and the construction-time invariants as editor smart constructors.

References

- [1] N. G. Leveson. *Engineering a Safer World: Systems Thinking Applied to Safety*. MIT Press, 2012. (And the *STPA Handbook*, Leveson & Thomas, 2018.)
- [2] J. Thomas. *Extending and Automating a Systems-Theoretic Hazard Analysis for Requirements Generation and Analysis*. PhD dissertation, MIT, 2013.
- [3] A. Abdulkhaleq and S. Wagner. *XSTAMPP: An eXtensible STAMP Platform as Tool Support for Safety Engineering*. STAMP Workshop, MIT, 2015.
- [4] A. Abdulkhaleq, S. Wagner, and N. Leveson. *A Comprehensive Safety Engineering Approach for Software-Intensive Systems Based on STPA*. Procedia Engineering 128:2–11, 2015.
- [5] A. Abdulkhaleq and S. Wagner. *An Automatic Safety-Based Test Case Generation Approach Based on Systems-Theoretic Process Analysis*. ACM TOSEM, 2016. (XSTPA, STPA Verifier, STPA TCGenerator.)
- [6] C. Innes, A. Ireland, Y. Lin, and S. Ramamoorthy. *Anticipating Accidents through Reasoned Simulation*. 1st Intl. Symp. on Trustworthy Autonomous Systems (TAS ’23), 2023. arXiv:2306.04499.
- [7] A. Dakwat and E. Villani. *System Safety Assessment Based on STPA and Model Checking*. Safety Science, 2018. (UPPAAL case study.)
- [8] J. Arias et al. *Towards Dynamic Consistency Checking in Goal-Directed Predicate Answer Set Programming*. 2021. arXiv:2110.12053. (Reachability/denial idiom.)
- [9] SE-Stuttgart. *XSTAMPP source repository*. <https://github.com/SE-Stuttgart/XSTAMPP>.
- [10] *STPAmaster*. <https://stpamaster.com/>.

Note: this review was compiled from web sources; specific factual claims should be verified against the cited primary documents before publication.